

8. Introduction

Every developer eventually meets a LeetCode problem that *looks* terrifying and turns out to be gentle underneath. **Maximize Active Section with Trade II** is exactly that kind of problem. It carries the dreaded "Hard" label, it throws range queries at you, and it wraps everything in an operation that sounds convoluted the first time you read it. And yet, once you see the shape hiding inside — a small run of `1`s hugged by `0`s on both sides — the entire thing unravels.

This problem matters because it tests three skills interviewers genuinely care about. First, **problem reduction**: can you strip away the intimidating wording and find the tiny core idea? Second, **precomputation and query design**: can you recognize that answering many questions cheaply requires paying an upfront cost once? Third, **boundary reasoning**: can you handle the two annoying edges of a window without breaking the clean middle? These are the same skills you need when you build a text editor that reflows lines, a genome scanner, or a system that packs data into bits. Range queries over strings are everywhere in real **Software Engineering**, which is why big companies keep asking questions shaped like this one.

If you are grinding the daily challenge or prepping for interviews, this is a high-leverage problem. Learn the **o-1-o sandwich** idea once, and you will reuse the run-decompose-then-range-max pattern for years. Take a breath. We will go slowly, build the intuition first, and only then write **Python**.

9. Problem Overview

You are given a binary string `s` of length `n`. Each character is either `1` (an **active** section) or `0` (an **inactive** section). You are also given a list of queries. Each query is a pair `[l, r]` describing a slice of the string from index `l` to index `r` inclusive.

For each query, you may perform **at most one trade**, and the trade only affects the chosen slice. A trade happens in two steps:

1. Take a **maximal run of `1`s that has `0`s pressing against it on both sides**, and turn that entire run into `0`s.
2. Then take a **maximal run of `0`s that now has `1`s on both sides**, and turn that entire run into `1`s.

After the trade, you report **the total number of `1`s in the whole string**, not just inside the slice.

The task: for each query, return the maximum number of active sections achievable.

Here is the single most important observation, and it is worth tattooing on your memory: **a trade never changes the total count of 1s by itself — it only rearranges them.** Erasing a run of 1s removes some ones; flooding the neighboring zeros adds them right back. So the net effect of any trade is purely a **merge of two zero-gaps around a lonely one-run**. That means:

$$\text{answer} = (\text{total number of 1s}) + (\text{best achievable gain in this window})$$

And the "gain" of a sandwich is simply the number of zeros on its left plus the number of zeros on its right. That is the whole problem.

10. Example

Let's make it concrete. Take the tiny string:

```
0 0 0 1 0
```

There is exactly one 1, sitting near the middle. It has three 0s on its left and one 0 on its right — zeros on **both** sides. That makes it a valid **0-1-0 sandwich**.

- **Step 1:** turn that run of 1s into 0s → 0 0 0 0 0.
- **Step 2:** the whole thing is now one solid run of zeros with 1s pressing in from the imaginary edges; fill it → 1 1 1 1 1.

We went from **one** active section to **five**. The single 1 we deleted was the *key* that unlocked the zeros on both of its sides. That is the intuition in one picture.

Now a slightly bigger one so the "total plus gain" formula clicks:

```
1 0 0 0 1 0 0
index: 0 1 2 3 4 5 6
```

There are already **two** 1s (at index 0 and index 4). The 1 at index 0 is *not* a sandwich — it only has zeros on its right, nothing on its left. But the 1 at index 4 has **three zeros on its left** (indices 1, 2, 3) and **two zeros on its right** (indices 5, 6). Its gain is $3 + 2 = 5$.

$$\text{answer} = \text{total ones (2)} + \text{best gain (5)} = 7$$

Trace it: delete the `1` at index 4 $\rightarrow$ `1 0 0 0 0 0 0`, then flood the big zero run bounded by the `1` at index 0 $\rightarrow$ `1 1 1 1 1 1 1`. Seven ones. The formula and the by-hand trace agree.

11. Intuition

How does an experienced engineer *discover* this instead of memorizing it? By asking, "What does the operation actually buy me?"

The operation is described in two dramatic steps, but conserve the total and the drama disappears. You are not gaining ones by "creating" them — you are gaining ones by **fusing two separate pools of zeros into a single fillable region**. The only zeros you are allowed to claim are the ones directly adjacent to a run of `1` s on *both* sides, because that middle run is the bridge that makes both gaps fillable at the same time.

So every candidate move maps to one shape: a run of `1` s with a run of `0` s on its left and a run of `0` s on its right. Name it the **Zero-One-Zero**. Its value is `left_zeros + right_zeros`. The best move in a window is the sandwich with the most bread.

Now layer on the queries. Naively, each query means "look inside this slice, find the best sandwich." But sandwiches don't move — a sandwich fully inside the window has *exactly* the gain we could measure once, upfront, for the whole string. The only sandwiches whose gain changes per query are the **two at the very edges**, because the window boundary can slice off part of their bread. Everything strictly in the middle is invariant.

That single insight — *only the two extreme sandwiches need special handling* — is what converts a slow per-query scan into two quick hand-checks plus one instant range-maximum lookup.

12. Brute Force Solution

Idea. For each query independently, rebuild the slice, break it into runs, scan for the best sandwich, and add its gain to the total number of ones.

Algorithm. 1. Precompute a prefix-sum of ones so the total is `O(1)` per query. 2. For a query `[l, r]`, walk `s[l..r]`, grouping consecutive equal characters into runs. 3. For every `1`-run that has a `0`-run on both sides *inside the window*, compute `left_zeros + right_zeros`. 4. Return `total_ones + max_gain` (or `total_ones` if no sandwich exists).

Advantages. - Dead simple to reason about and to write correctly. - An excellent **oracle**: I genuinely write this first and use it to differential-test the fast version against random inputs. It is cheap

insurance against off-by-one bugs.

Disadvantages. - It re-does all the run work for every single query. - With large n and many queries it times out.

Complexity. Let n be the string length and q the number of queries. Each query costs $O(n)$ in the worst case, so total is $O(n \cdot q)$ time. With $n = q = 10^5$, that is 10^{10} operations — far too slow for a Hard problem's limits. Space is $O(n)$.

13. Optimal Solution

The fast approach pays once, upfront, then answers each query almost instantly.

Step 1 — Decompose into runs. Walk the string a single time, grouping equal characters. Store each run's character plus its **absolute** inclusive start and end indices. Absolute indices matter: during a query we compare them directly against l and r with no offset math. Count the total ones here too, since it never changes again.

Step 2 — Precompute every sandwich's gain. Scan the runs. Whenever a 1 -run has a 0 -run immediately before **and** after it, record a candidate. For each candidate store:

Field	Meaning
a, b	start / end index of the 1 -run (the filling)
c	start index of the left 0 -run (outer left edge)
d	end index of the right 0 -run (outer right edge)
gain	$(a - c) + (d - b) \rightarrow$ left zeros + right zeros

Because we emit candidates left-to-right, their a and b values come out **already sorted**. That's what lets a query use binary search directly, no separate sort needed.

Step 3 — Build a sparse table over the gains. A sparse table is a precomputed "cheat sheet" that answers *range maximum* in $O(1)$. We choose it over a segment tree because the gains are **static** and $\max$ is **idempotent** — no lazy propagation, no updates, just fast queries.

Step 4 — Answer each query. For $[l, r]$:

1. Use binary search to find the candidates whose 1 -run sits **strictly interior** to the window. In practice: bisect on $l+1$ and $r-1$ to grab indices $i_0 \dots i_1$.
2. If $i_0 > i_1$, no valid sandwich fits $\rightarrow$ answer is `total_ones`.
3. The two **edge** candidates (i_0 and i_1) might have their zero-bread clipped by the boundary, so **recompute** their gains with clamped edges: left zeros become $a - \max(c, l)$, right zeros

become `min(d, r) - b`.

4. Every candidate **strictly between** `i0` and `i1` is fully inside the window, so the sparse table hands us their maximum gain instantly.
5. Answer = `total_ones + max(clipped_left_gain, clipped_right_gain, interior_max)`.

Slow to build, lightning to ask.

14. Python Code

```
from bisect import bisect_left, bisect_right

class Solution:
    def maxActiveSectionsAfterTrade(self, s: str, queries):
        n = len(s)
        total_ones = s.count("1")

        # Step 1: decompose into runs -> (char, start, end) inclusive.
        runs = []
        i = 0
        while i < n:
            j = i
            while j < n and s[j] == s[i]:
                j += 1
            runs.append((s[i], i, j - 1))
            i = j

        # Step 2: every 0-1-0 sandwich. a,b = ones run; c,d = outer zero edges.
        cand_a, cand_b, cand_c, cand_d, gain = [], [], [], [], []
        for k in range(1, len(runs) - 1):
            ch, a, b = runs[k]
            if ch != "1":
                continue
            _, c, _ = runs[k - 1]      # left zero run start
            _, _, d = runs[k + 1]      # right zero run end
            cand_a.append(a)
            cand_b.append(b)
            cand_c.append(c)
            cand_d.append(d)
            gain.append((a - c) + (d - b))

        # Step 3: sparse table over gain for O(1) range max.
        sparse = self._build_sparse(gain)

        # Step 4: answer queries.
        ans = []
        for l, r in queries:
```

```

        best = total_ones + self._query(
            l, r, cand_a, cand_b, cand_c, cand_d, gain, sparse
        )
        ans.append(best)
    return ans

@staticmethod
def _build_sparse(arr):
    m = len(arr)
    if m == 0:
        return []
    LOG = m.bit_length()
    table = [arr[:]]
    length = 1
    for _ in range(1, LOG):
        prev = table[-1]
        nxt = [
            max(prev[i], prev[i + length])
            for i in range(m - 2 * length + 1)
        ]
        if not nxt:
            break
        table.append(nxt)
        length *= 2
    return table

@staticmethod
def _range_max(table, lo, hi):          # inclusive [lo, hi]
    k = (hi - lo + 1).bit_length() - 1
    return max(table[k][lo], table[k][hi - (1 << k) + 1])

def _query(self, l, r, A, B, C, D, gain, sparse):
    # Candidates whose ones-run is strictly interior to [l, r].
    i0 = bisect_left(A, l + 1)          # a >= l + 1
    i1 = bisect_right(B, r - 1) - 1     # b <= r - 1
    if i0 > i1:
        return 0

    # Clip the two edge candidates against the window boundary.
    left = (A[i0] - max(C[i0], l)) + (min(D[i0], r) - B[i0])
    right = (A[i1] - max(C[i1], l)) + (min(D[i1], r) - B[i1])
    best = max(left, right)

    # Interior candidates keep their full precomputed gain.
    if i0 + 1 <= i1 - 1:
        best = max(best, self._range_max(sparse, i0 + 1, i1 - 1))
    return best

def _demo():
    sol = Solution()
    # 1 0 0 0 1 0 0 : full window -> 2 ones + gain 5 = 7
    assert sol.maxActiveSectionsAfterTrade("1000100", [[0, 6]]) == [7]

```

```
# 0 0 0 1 0 : full window -> 1 one + gain 4 = 5
assert sol.maxActiveSectionsAfterTrade("00010", [[0, 4]]) == [5]
# no sandwich -> just the ones
assert sol.maxActiveSectionsAfterTrade("111", [[0, 2]]) == [3]
# window slicing through a 1-run disqualifies it
assert sol.maxActiveSectionsAfterTrade("1000100", [[2, 6]]) == [3]
print("all good")

if __name__ == "__main__":
    _demo()
```

15. Code Walkthrough

- `total_ones = s.count("1")` — computed once, because a trade never changes it. This is the invariant the whole solution leans on.
- **Run decomposition** (`while i < n`) — the classic single-pass grouping. Each run stores its character and *absolute inclusive* endpoints, so later comparisons against `l` and `r` need no arithmetic gymnastics.
- **Candidate loop** (`for k in range(1, len(runs) - 1)`) — we start at `1` and stop before the last run, because a sandwich needs a neighbor on both sides. We only keep `1`-runs (`if ch != "1": continue`). `c` is the left zero run's start, `d` is the right zero run's end. The gain `(a - c) + (d - b)` counts left zeros plus right zeros.
- **_build_sparse** — standard sparse table for range maximum. Each level `table[k]` holds the max over windows of length 2^k . Building is $O(m \log m)$.
- **_range_max** — the idempotent `max` trick: two overlapping power-of-two windows cover any range, so overlap doesn't matter. $O(1)$ per lookup.
- **_query binary search** — `bisect_left(A, l + 1)` finds the first candidate whose `1`-run starts at `l+1` or later; `bisect_right(B, r - 1) - 1` finds the last whose `1`-run ends at `r-1` or earlier. The `l+1 / r-1` bounds enforce **strict interiority**, which is what makes a window slicing through a `1`-run correctly disqualify it.
- **Edge clipping** — `max(C[i0], l)` and `min(D[i0], r)` clamp the outer zero edges to the window, so a sandwich hanging past the boundary only gets credit for the bread actually inside.

16. Dry Run

Let's fully trace `s = "1000100"` with query `[0, 6]`.

Runs: `('1', 0, 0)`, `('0', 1, 3)`, `('1', 4, 4)`, `('0', 5, 6)`.

Total ones: 2 .

Candidates (index k from 1 to 2): - $k = 1 \rightarrow \text{char } '0'$, skip. - $k = 2 \rightarrow \text{char } '1'$, $a=4$, $b=4$; left run $('0', 1, 3) \rightarrow c=1$; right run $('0', 5, 6) \rightarrow d=6$. Gain = $(4 - 1) + (6 - 4) = 3 + 2 = 5$.

So $A = [4]$, $B = [4]$, $C = [1]$, $D = [6]$, gain = $[5]$.

Query $[0, 6]$:- $i0 = \text{bisect_left}([4], 1) = 0$ - $i1 = \text{bisect_right}([4], 5) - 1 = 1$ - $1 = 0$ - $i0(0) \leq i1(0) \rightarrow \text{proceed}$. - Edge clip for candidate 0: $\text{left_zeros} = 4 - \max(1, 0) = 3$; $\text{right_zeros} = \min(6, 6) - 4 = 2$; gain = 5. - No strictly-interior candidates ($i0+1 > i1-1$). - best = 5.

Answer: total_ones (2) + 5 = 7. ✓ Matches our hand trace.

17. Complexity Analysis

Let n be the string length and q the number of queries.

Phase	Time	Space
Run decomposition	$O(n)$	$O(n)$
Candidate + gain precompute	$O(n)$	$O(n)$
Sparse table build	$O(n \log n)$	$O(n \log n)$
Per query (2 bisections + 1 range max)	$O(\log n)$	$O(1)$
Total	$O(n \log n + q \log n)$	$O(n \log n)$

Why these are correct. Each character is visited once during decomposition, so runs and candidates are linear. The sparse table has $\log n$ levels, each of size at most n . A query does two binary searches ($O(\log n)$) and one constant-time range maximum, and edge clipping is $O(1)$. This comfortably beats the brute force's $O(n \cdot q) = 10^4 \cdot 10^6$ collapses to roughly $n \log n + q \log n \approx 10^5 \cdot 17 \cdot 2 \approx 3.4 \times 10^6$ operations.

18. Alternative Solutions

- **Sparse table vs. segment tree.** A segment tree also answers range maximum, in $O(\log n)$ per query and $O(n)$ space. But because our gains are **static** and max is idempotent, the sparse table gives $O(1)$ queries with a simpler implementation and no lazy propagation. Reach for the segment tree only if the string can **mutate between queries** — then the sparse table is dead, and you switch to a segment tree over candidate gains with $O(\log n)$ updates, re-deriving candidates around the edit.

- **Offline sorting / Mo's algorithm.** You *could* sort queries and sweep, but there's no need here — the per-query work is already $O(\log n)$ and independent, so an online sparse table is both simpler and faster.

The sparse table is the honest winner for this exact problem: simplest structure that hits the required complexity.

19. Edge Cases

- **No sandwich exists** (e.g. "111" or "000"): the candidate list is empty or $i_0 > i_1$, so the gain is 0 and the answer is just `total_ones`.
- **Window slices through a 1-run**: the $l+1 / r-1$ bisect bounds exclude any 1-run touching the boundary, so it correctly stops qualifying.
- **Tiny window** (single character, or $l == r$): $i_0 > i_1$ triggers, returning `total_ones` safely.
- **Empty or all-same string**: handled by the empty candidate path.
- **Sandwich hanging past both edges**: clamping with $\max(C, l)$ and $\min(D, r)$ credits only the bread inside the window.
- **Maximum-size input**: the $O(n \log n)$ build and $O(\log n)$ queries keep you well within limits.

20. Common Mistakes

1. **Recounting the ones per query.** The total is invariant — count it once. Recomputing it inside each query is wasted work and invites bugs.
2. **Forgetting to clip the edge sandwiches.** The two extreme candidates can hang past the window; using their full precomputed gain over-counts.
3. **Off-by-one in the bisect bounds.** It must be $l+1$ and $r-1$ (strict interiority), plus the -1 after `bisect_right`. Mixing these up either includes boundary-touching runs or drops valid ones.
4. **Treating a leading/trailing 1-run as a candidate.** A run at the string's very edge has zeros on only one side and must be excluded — hence the loop runs `k` from 1 to `len(runs) - 2`.
5. **Building a segment tree with lazy propagation you never use.** For static max, the sparse table is simpler; the extra machinery is dead flexibility.
6. **Assuming the answer lives only inside the window.** The problem asks for total active sections across the *whole* string, not just the slice.

21. Interview Questions

- **"Why is the total number of ones invariant under a trade?"** Because erasing a `1`-run and flooding the neighboring zeros refunds exactly the ones you removed — the net effect is only the zero-gap merge.
- **"Why does only the leftmost and rightmost sandwich need clipping?"** Every candidate strictly between them has both flanking zero-runs fully inside `[1, r]`, so its precomputed gain is exact.
- **"What if the string can change between queries?"** Sparse table dies; switch to a segment tree over gains with $O(\log n)$ updates and re-derive candidates around the edit.
- **"Why a sparse table over a segment tree here?"** Static data + idempotent `max` = $O(1)$ queries, no lazy propagation.
- **"How would you test the fast solution?"** Keep the $O(n)$ brute force as an oracle and differential-test on random inputs — cheap insurance against off-by-one bugs.

22. Similar Problems

- **Maximize Active Section with Trade I** — the single-string, no-query version; solving it first builds the sandwich intuition you reuse here.
- **Range Sum Query - Immutable / Mutable (LeetCode 303 / 307)** — the same precompute-once, query-cheap philosophy, with sparse-table vs. segment-tree as the static/mutable fork.
- **Sparse Table Range Minimum/Maximum problems** — direct practice for the $O(1)$ range-max structure.
- **Max Consecutive Ones III (LeetCode 1004)** — another "flip zeros to grow a run of ones" theme, sliding-window flavored.
- **Count Binary Substrings (LeetCode 696)** — run decomposition of a binary string as the core move.

They're related because each rewards spotting a **run structure** and answering **range questions** with a static precomputed metric.

23. Key Takeaways

- A Hard label often hides one small shape. Here it's the **o-1-o sandwich**.
- **Conserve the invariant:** total ones never change, so `answer = total_ones + best_gain`.

- **Decompose into runs**, precompute each sandwich's gain, and let a **sparse table** serve the middle in $O(1)$.
- Only the **two edge sandwiches** need clipping — everything between them is exact.
- Keep the $O(n)$ brute force as an **oracle** for testing the fast path.

This is the run-decompose-then-range-max pattern, and it generalizes to a whole family of interval problems.

24. Watch the Video

Watching the trace happen in real time makes the sandwich *stick*. In the video we trace `00010 → 11111` by hand, predict answers together, compare brute force against the fast approach, and walk through the exact edge cases that trip people up. If a step here felt abstract, seeing it move on screen will make it click.

 **Watch the full three-level walkthrough here:** {LINK}

25. About the Series

This is part of the **Daily Python LeetCode** series — one problem a day, explained the way a senior engineer would teach a teammate. No fluff, no "just memorize it." We build the intuition first, then write clean, production-quality Python, then dry-run it until it's obvious. Every problem covers brute force, the optimal approach, complexity, edge cases, and interview follow-ups, so you leave understanding the *why*, not just the answer.

26. Call To Action

If this helped the Hard label feel a little smaller:

-  **Subscribe on YouTube** so you never miss a daily walkthrough.
-  **Subscribe on Substack** to get each solution and article in your inbox.
-  **Comment** with your own dry run of `1000100` — did you land on 7?
-  **Share** this with a friend who's grinding the daily challenge.

See you in tomorrow's problem, where this same run-plus-range-max trick cracks a genuinely nastier interval puzzle.

Quick Review

String toggle puzzle: one contiguous flip to maximize same-value run. Pattern?

Sliding-window / merge-adjacent-segments. Tell: 'flip one substring' + maximize contiguous count
→ scan segment lengths, merge neighbors separated by a small gap.

Time complexity of the segment-scan solution?

$O(n)$ — one pass to build runs, one pass to evaluate merges. n = string length.

Space complexity?

$O(n)$ if you store the run-length list; $O(1)$ achievable by tracking only the last few segment lengths while streaming.

Trickiest bug in Maximize Active Section II?

Off-by-one at string ends and mishandling all-same strings: no inactive gap to trade, so the answer is just the existing active count.

This vs classic 'Max Consecutive Ones III'?

Ones III flips up to k zeros anywhere (budget window). Here one whole block moves/merges — you combine two active runs across a single gap, not flip k bits.

Follow-up: what if many trades allowed instead of one?

Problem collapses — you could gather all active sections together, so the answer becomes the total active count regardless of arrangement.